

Lidar on an FPGA

A simple, hands-on guide to reading a lidar sensor
with the Zybo Z7-10 board

Muhammad Refansa · FPGA study group · October 2026

ABOUT THIS GUIDE

This guide explains one finished project from start to end. We connected a spinning laser distance sensor (an RPLIDAR A1) directly to a Zybo Z7-10 board. Some parts of the system live inside the FPGA: a serial port, a pin controller, and a small speed controller that we wrote in Verilog. The rest is a C program running on the board's ARM processor. It reads the sensor's data and prints the results. We wrote this for friends who are learning FPGA with us. You do not need to know much already. Each chapter explains the idea in simple words, shows the real code or numbers from our project, and ends with a few questions. Everything here was measured on the real board or checked with a script. When we did not prove something, we say so. The full code is in the public repository [Rennaafi/rplidar-zybo](#).

Contents

How to use this guide	4
Small notes before you start	4
1 The Big Picture	5
1.1 What we built	5
1.2 Why this is a good project to learn from	6
1.3 The result in one screen	6
2 Background You Need	9
2.1 The Zynq: two computers in one chip	9
2.2 Memory-mapped peripherals	9
2.3 UART: sending bytes on one wire	10
2.4 PWM: controlling speed by switching fast	10
2.5 Programming without an operating system	10
3 The Sensor: Talking to the RPLIDAR A1	11
3.1 What it does	11
3.2 The connector	11
3.3 The protocol (the rules of the conversation)	11
3.4 What is inside one node	12
3.5 One full turn	12
4 The Hardware (the FPGA Side)	14
4.1 Wiring and pins	14
4.2 The block design	14
4.3 The PWM block, line by line	15
4.4 Proving it in simulation	15
4.5 Timing and the build result	16
5 The Software, Part 1: Talking to Registers	17
5.1 How <code>main.c</code> is organised	17
5.2 Reading the serial port without waiting	17
5.3 Address fallbacks	18
5.4 The timer that did not start	18
6 The Software, Part 2: Not Losing Bytes	19
6.1 The problem	19
6.2 The solution: a ring buffer between the program and the console	19
6.3 The console is also a limit	20
6.4 Proof, not hope	20
7 The Software, Part 3: Starting the Lidar and the Watchdog	21
7.1 The start-up steps	21
7.2 Time limits with one shared deadline	21
7.3 The watchdog	22
7.4 Console keys	22
8 The Parser: Making Sense of the Byte Stream	23
8.1 The problem	23
8.2 Step 1: the sliding window	23
8.3 Step 2: one good window is not enough	23
8.4 Step 3: building one turn	24
8.5 Tested on the PC first	24
9 Test Strategy: Finding Which Half Is Broken	25
9.1 The test ladder	25

9.2 The PC tools	25
10 Build and Run It Yourself	26
10.1 What you need	26
10.2 Step by step	26
10.3 Problems we had (so you can avoid them)	26
11 Results and What They Teach	27
11.1 The motor speed test	27
11.2 What the numbers tell us	27
11.3 Angle between points versus one-degree bins	28
12 Bug Diary: What Went Wrong and How We Found It	29
13 Exercises	30
14 Where This Goes Next	31
Appendix A: Glossary	32
Appendix B: Answers	33
Appendix C: Quick Reference	34

How to use this guide

If you are new to the Zynq chip: read chapters 1 and 2 slowly. Skim the rest. Then try the exercises in chapter 13.

If you already used Vivado: read chapters 3 to 8. Then rebuild the project with chapter 10.

If you only want to run it: go straight to chapter 10. The other chapters explain what happens when you do.

THINK OF IT LIKE THIS

The best habit in this project was not clever code. It was **testing each half on its own** before joining them. When something failed, we always asked: “Which half is the problem?” Chapter 9 shows how we did this.

Small notes before you start

- This font means code, a file name, or a register (a memory spot inside a chip).
- Numbers like `0x42C00000` are written in hexadecimal (base 16), the way C code writes them.
- **Think of it like this** boxes give an everyday picture. **Try it yourself** boxes are small experiments.
- **PS** means “processing system”: the ARM processor part of the chip. **PL** means “programmable logic”: the FPGA part. You will meet both in chapter 2.

1 The Big Picture

1.1 What we built

The RPLIDAR A1 is a small laser sensor. It spins around about 6 to 7 times every second. While it spins, it measures the distance to the objects around it, about 2000 times per second. Think of it as a very fast measuring tape that points in every direction.

We connected it to the Zybo Z7-10 board and made the board do these things:

1. Start the sensor and check that it is healthy.
2. Spin its motor at a speed we choose. The speed signal is made inside the FPGA.
3. Receive the stream of measurements and find where each full turn starts.
4. Print one line of results for every turn, light four LEDs depending on what is in front of the sensor, and (if we want) send the whole 360° picture to a PC that draws it live.



Figure 1: The real setup. The lidar (bottom) connects through a breadboard to the Pmod JE connector on the Zybo. The blue cable is the board's USB cable. It gives power and a text console.

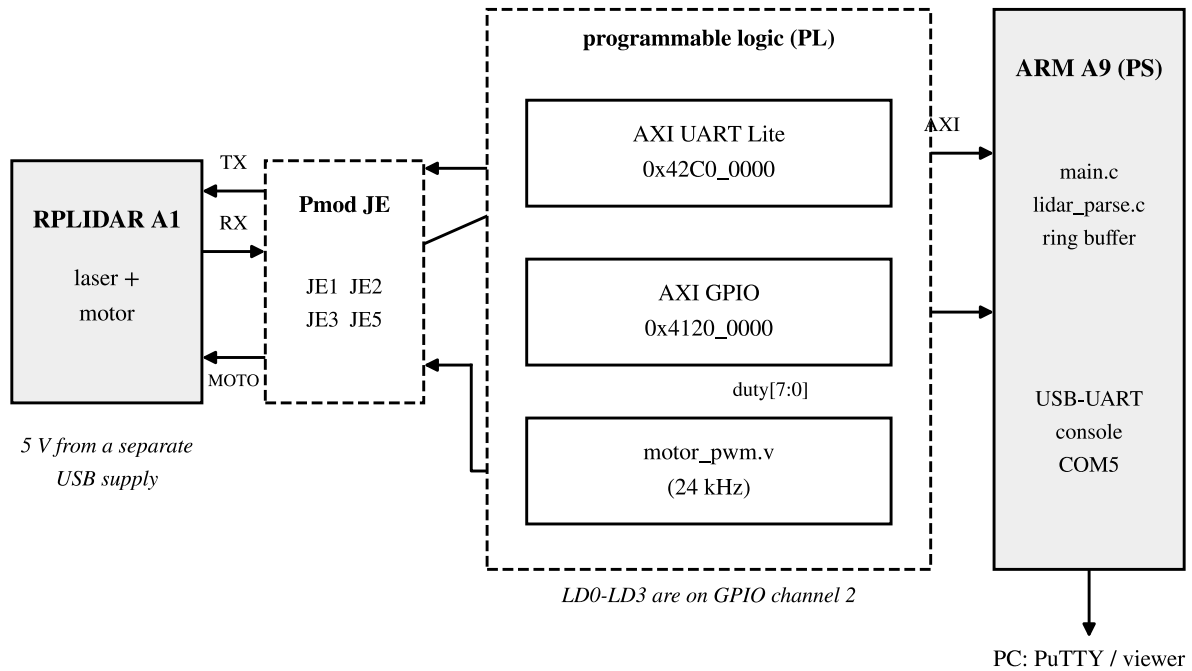


Figure 2: The whole system. The dashed box is the FPGA part that you can see in Vivado. The grey box on the right is the ARM processor that runs our C program.

1.2 Why this is a good project to learn from

It touches many beginner topics in one small system. Each topic also breaks in a clear way if you get it wrong:

Topic	Where you see it	What goes wrong if you ignore it
Pins and voltage	Pmod JE wiring (ch. 4)	No data, or a damaged pin
Block design and AXI	UART and GPIO inside the FPGA (ch. 4)	The processor cannot “see” the parts
Hardware design and simulation	<code>motor_pwm.v</code> and its test (ch. 4)	The motor runs at the wrong speed
Programming without an OS	<code>main.c</code> (ch. 5-7)	The program freezes (see the timer story)
Timing and speed	16-byte buffer, ring buffer (ch. 6)	Data is lost without any warning
Reading a data format	<code>lidar_parse.c</code> (ch. 8)	Fake “scans” made from noise
How to test	loopback, fake source (ch. 9)	Days lost guessing what is broken

Table 1: Topics in this project.

1.3 The result in one screen

With the lidar wired straight to the Zybo (no PC in the middle), the console prints one line per turn:

```
COM5 - PuTTY
rev 405 7.07 Hz pts 280 valid 204 near 123 mm 0 17 deg front 123 mm duty 200 ovr 0 sync 15
rev 406 7.10 Hz pts 279 valid 204 near 123 mm 0 47 deg front 124 mm duty 200 ovr 0 sync 15
rev 407 7.09 Hz pts 279 valid 204 near 123 mm 0 48 deg front 124 mm duty 200 ovr 0 sync 15
rev 408 7.07 Hz pts 280 valid 195 near 123 mm 0 17 deg front 123 mm duty 200 ovr 0 sync 15
rev 409 7.09 Hz pts 279 valid 182 near 123 mm 0 15 deg front 123 mm duty 200 ovr 0 sync 15
rev 410 7.07 Hz pts 280 valid 180 near 124 mm 0 12 deg front 124 mm duty 200 ovr 0 sync 15
rev 411 7.07 Hz pts 280 valid 183 near 123 mm 0 13 deg front 123 mm duty 200 ovr 0 sync 15
rev 412 7.07 Hz pts 280 valid 204 near 123 mm 0 50 deg front 124 mm duty 200 ovr 0 sync 15
rev 413 7.04 Hz pts 281 valid 214 near 124 mm 0 49 deg front 125 mm duty 200 ovr 0 sync 15
rev 414 7.07 Hz pts 280 valid 203 near 123 mm 0 47 deg front 125 mm duty 200 ovr 0 sync 15
rev 415 7.07 Hz pts 280 valid 210 near 123 mm 0 46 deg front 126 mm duty 200 ovr 0 sync 15
rev 416 7.04 Hz pts 281 valid 208 near 124 mm 0 44 deg front 127 mm duty 200 ovr 0 sync 15
rev 417 7.07 Hz pts 280 valid 215 near 124 mm 0 35 deg front 128 mm duty 200 ovr 0 sync 15
rev 418 7.07 Hz pts 280 valid 210 near 125 mm 0 35 deg front 130 mm duty 200 ovr 0 sync 15
rev 419 7.07 Hz pts 280 valid 212 near 124 mm 0 147 deg front 129 mm duty 200 ovr 0 sync 15
rev 420 7.07 Hz pts 280 valid 214 near 125 mm 0 37 deg front 129 mm duty 200 ovr 0 sync 15
rev 421 7.04 Hz pts 281 valid 209 near 125 mm 0 36 deg front 128 mm duty 200 ovr 0 sync 15
rev 422 7.07 Hz pts 280 valid 209 near 124 mm 0 147 deg front 128 mm duty 200 ovr 0 sync 15
rev 423 7.07 Hz pts 280 valid 217 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 424 7.07 Hz pts 280 valid 215 near 124 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 425 7.07 Hz pts 280 valid 214 near 126 mm 0 37 deg front 129 mm duty 200 ovr 0 sync 15
rev 426 7.07 Hz pts 280 valid 215 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 427 7.09 Hz pts 279 valid 215 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 428 7.07 Hz pts 280 valid 208 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 429 7.07 Hz pts 280 valid 209 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 430 7.09 Hz pts 279 valid 213 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 431 7.07 Hz pts 280 valid 208 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 432 7.10 Hz pts 279 valid 214 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 433 7.09 Hz pts 279 valid 214 near 124 mm 0 36 deg front 128 mm duty 200 ovr 0 sync 15
rev 434 7.09 Hz pts 279 valid 208 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 435 7.07 Hz pts 280 valid 208 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 436 7.09 Hz pts 279 valid 210 near 124 mm 0 38 deg front 128 mm duty 200 ovr 0 sync 15
rev 437 7.07 Hz pts 280 valid 215 near 124 mm 0 38 deg front 128 mm duty 200 ovr 0 sync 15
rev 438 7.07 Hz pts 280 valid 214 near 124 mm 0 39 deg front 127 mm duty 200 ovr 0 sync 15
rev 439 7.09 Hz pts 279 valid 207 near 123 mm 0 38 deg front 127 mm duty 200 ovr 0 sync 15
rev 440 7.07 Hz pts 280 valid 210 near 123 mm 0 42 deg front 126 mm duty 200 ovr 0 sync 15
rev 441 7.07 Hz pts 280 valid 200 near 124 mm 0 18 deg front 124 mm duty 200 ovr 0 sync 15
rev 442 7.07 Hz pts 280 valid 193 near 123 mm 0 29 deg front 123 mm duty 200 ovr 0 sync 15
rev 443 7.07 Hz pts 280 valid 192 near 123 mm 0 16 deg front 123 mm duty 200 ovr 0 sync 15
rev 444 7.07 Hz pts 280 valid 172 near 123 mm 0 7 deg front 123 mm duty 200 ovr 0 sync 15
rev 445 7.07 Hz pts 276 valid 172 near 123 mm 0 56 deg front 124 mm duty 200 ovr 0 sync 20
rev 446 7.07 Hz pts 280 valid 167 near 123 mm 0 41 deg front 123 mm duty 200 ovr 0 sync 20
rev 447 7.07 Hz pts 280 valid 157 near 123 mm 0 353 deg front 123 mm duty 200 ovr 0 sync 20
rev 448 7.07 Hz pts 280 valid 161 near 124 mm 0 58 deg front 125 mm duty 200 ovr 0 sync 20
rev 449 7.09 Hz pts 279 valid 162 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 450 7.07 Hz pts 280 valid 163 near 123 mm 0 353 deg front 123 mm duty 200 ovr 0 sync 20
rev 451 7.10 Hz pts 279 valid 161 near 123 mm 0 58 deg front 125 mm duty 200 ovr 0 sync 20
rev 452 7.07 Hz pts 280 valid 163 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 453 7.09 Hz pts 279 valid 166 near 123 mm 0 58 deg front 123 mm duty 200 ovr 0 sync 20
rev 454 7.12 Hz pts 278 valid 161 near 123 mm 0 58 deg front 123 mm duty 200 ovr 0 sync 20
rev 455 7.12 Hz pts 278 valid 158 near 123 mm 0 59 deg front 123 mm duty 200 ovr 0 sync 20
rev 456 7.12 Hz pts 278 valid 161 near 123 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 457 7.12 Hz pts 278 valid 159 near 123 mm 0 60 deg front 123 mm duty 200 ovr 0 sync 20
rev 458 7.12 Hz pts 278 valid 161 near 124 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 459 7.15 Hz pts 277 valid 160 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 460 7.12 Hz pts 278 valid 162 near 123 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 461 7.12 Hz pts 278 valid 160 near 124 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 462 7.15 Hz pts 277 valid 159 near 123 mm 0 58 deg front 124 mm duty 200 ovr 0 sync 20
rev 463 7.12 Hz pts 278 valid 167 near 123 mm 0 355 deg front 123 mm duty 200 ovr 0 sync 20
rev 464 7.12 Hz pts 278 valid 156 near 123 mm 0 57 deg front 124 mm duty 200 ovr 0 sync 20
rev 465 7.15 Hz pts 277 valid 160 near 123 mm 0 55 deg front 124 mm duty 200 ovr 0 sync 20
```

Figure 3: Console output (PuTTY on COM5). About 7.1 turns per second and 280 points per turn. `ovr 0` means zero lost bytes, over hundreds of turns. This run used duty 200. The default in the final code is 120.

A PC program can also draw the same data as a round plot:

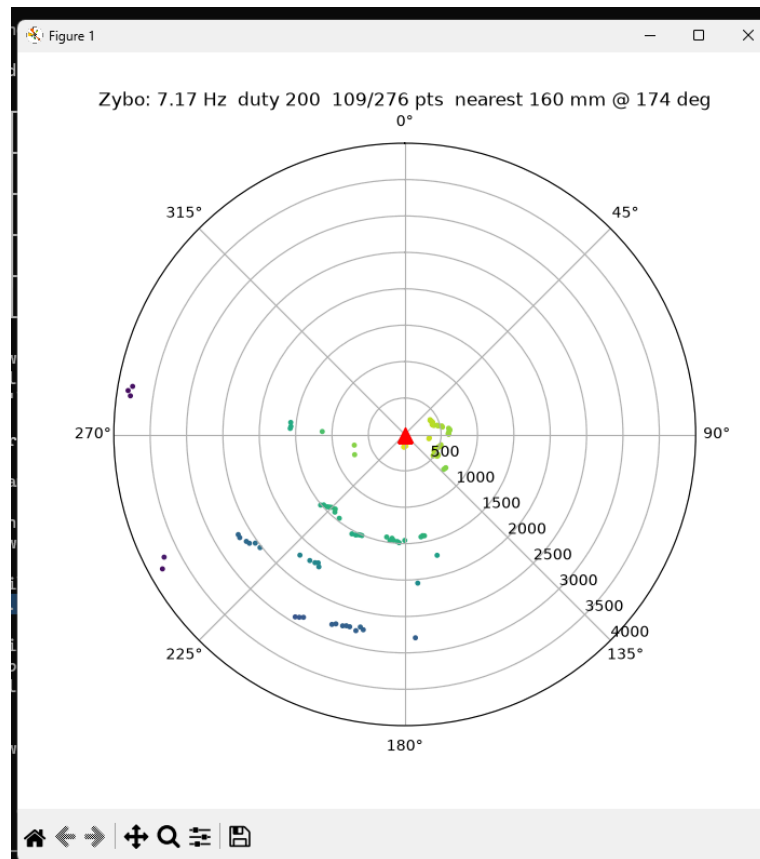


Figure 4: Live plot fed by the Zybo. The red triangle is the sensor. 0° is straight ahead. The rings show distance in millimetres.

THINK OF IT LIKE THIS

A lidar scan is only a long list of pairs: *(angle, distance)*. Each pair is one laser shot. This whole project is about moving that list from the spinning sensor into the board's memory, correctly, without losing any of it.

CHECK YOURSELF (answers in Appendix B)

Q1.1 What are the two halves of the Zynq chip called? Which half runs `main.c`?

Q1.2 Which part of the project is written in Verilog, and which part in C?

2 Background You Need

This chapter explains the basic ideas. Take your time here. The rest of the guide becomes easy after this.

2.1 The Zynq: two computers in one chip

The chip on the Zybo has two different parts:

- The **PS** (processing system) is a normal ARM processor, like a small computer. It has ready-made parts such as a memory controller, USB, Ethernet and serial ports. You do not need to build it. It is always there.
- The **PL** (programmable logic) is the FPGA part. It is empty when the board starts. You fill it by loading a file called a **bitstream**. Anything you design, such as a serial port or a speed controller, lives here.

The two parts talk through a “road” called **AXI**. In our design, the ARM is the boss (the *master*). Our PL blocks are helpers (the *slaves*). The ARM reads and writes small memory spots inside the helpers. The PS also gives a clock signal to the PL (called **FCLK_CLK0**). We set it to 100 MHz.

THINK OF IT LIKE THIS

The PS is a brain that already exists. The PL is a box of Lego bricks. We choose which bricks to add, and we give each brick an address so the brain can find it.

2.2 Memory-mapped peripherals

For the ARM, every helper part is just a set of addresses. If the ARM reads the address **0x42C00008**, it does not read normal memory. It reads the *status* of our serial port block. If the ARM writes to **0x41200000**, it does not store data. It changes 8 wires coming out of our GPIO block. Those wires set the motor speed.

In C, this is done with two simple functions:

```
u32 v = Xil_In32(addr);    // read a 32-bit register
Xil_Out32(addr, v);        // write a 32-bit register
```

All the “driver” functions in this project (**XUartLite_ReadReg** and similar) just call these two functions. That is why they work in every version of Vitis.

Block	Base address	Offset	What this register does
AXI UART Lite	0x42C00000	+0x0	Read a received byte (RX FIFO)
		+0x4	Write a byte to send (TX FIFO)
		+0x8	Status (bit 0: a byte is waiting, bit 5: a byte was lost)
		+0xC	Control (clear the buffers)
AXI GPIO	0x41200000	+0x0	Channel 1: motor speed (8 bits)
		+0x8	Channel 2: the LEDs (4 bits)
PS UART1	0xE0001000		USB text console to the PC (part of the PS)

Table 2: Everything the C program talks to. The first two base addresses are set by **build_hw.tcl**.

2.3 UART: sending bytes on one wire

A **UART** is a simple way to send bytes over one wire. There is no clock wire. Instead, both sides agree on a speed in advance. This speed is called the **baud rate**. The wire stays high when nothing is sent. Each byte is sent like this:

idle	start (0)	D0	D1	D2	D3	D4	D5	D6	D7	stop (1)
------	--------------	----	----	----	----	----	----	----	----	-------------

At 115200 baud, one bit takes 8.68 μs . One byte uses 10 bits (1 start, 8 data, 1 stop). So the wire can carry at most **11 520 bytes per second**. Two rules to remember:

- One side's TX (transmit) goes to the other side's RX (receive). Mixing them up is the most common wiring mistake.
- Both devices need a shared ground wire, and both must use the same voltage level (here 3.3 V).

2.4 PWM: controlling speed by switching fast

You can control a motor's speed without a smooth voltage. Switch the power fully on, then fully off, very fast. The motor feels the average. The part of the time that the signal is on is called the **duty cycle**.

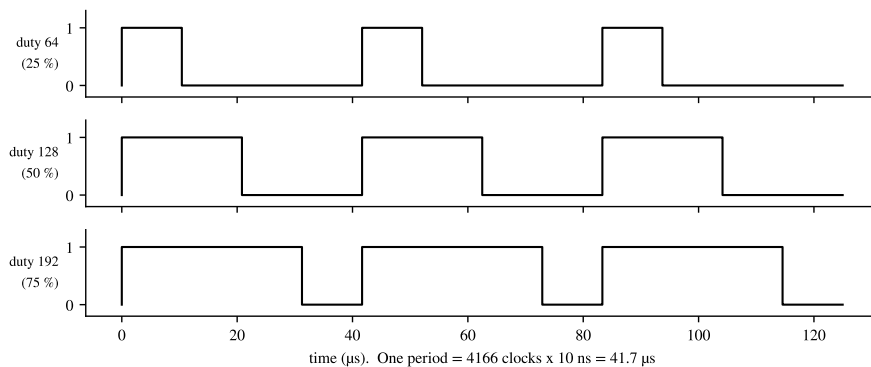


Figure 5: Three duty values at 24 kHz. The pattern repeats every 41.7 μs . Only the time spent high changes.

The lidar has a pin called MOTOCTL that accepts exactly this kind of signal.

THINK OF IT LIKE THIS

PWM is a counter running around a loop, plus a small check that says: “Is the counter below my limit? Then output 1.” That is the whole circuit. You will read it in about 15 lines of Verilog in chapter 4.

2.5 Programming without an operating system

We call this **bare-metal**. Our C program runs directly on the ARM. There is no Windows or Linux. There is no scheduler and no threads. The `sleep` command does not let other tasks run. The program is one endless loop, and everything must happen inside that loop, quickly. This is the reason chapter 6 exists.

CHECK YOURSELF (answers in Appendix B)

Q2.1 At 115200 baud, how many microseconds does one byte take on the wire?

Q2.2 The ARM writes to `0x41200000`. Which pins change, and through which block?

Q2.3 Why can you not simply use `sleep(1)` to wait, while still reading the serial port?

3 The Sensor: Talking to the RPLIDAR A1

3.1 What it does

Inside the sensor, a laser and a small camera sit on a platform that a motor spins. About 2000 times per second, it fires the laser, works out the distance, and reports it together with the current angle. After one full turn, it starts again. We do not control the laser. We only control two things: the motor speed (with PWM), and whether the sensor starts or stops sending data.

3.2 The connector

Signal	What it is for
TX	Data from the sensor to us (3.3 V, 115200 baud)
RX	Commands from us to the sensor
MOTOCTL	PWM input that sets the motor speed (we make it in the FPGA)
VS5.0, VMOTO	5 V power for the electronics and for the motor
GND	Ground, shared with everything else

Table 3: The lidar’s pins as we use them.

The motor needs much more current than a Pmod pin can give. Also, the Pmod only gives 3.3 V. So the lidar gets its 5 V from a separate USB power source. Only the *signals* and the *ground* go to the Zybo.

3.3 The protocol (the rules of the conversation)

Every request is two bytes: first A5, then a command byte. These are the commands we use:

Command	Bytes	What the sensor does
STOP	A5 25	Stops scanning. No reply.
RESET	A5 40	Restarts the sensor. No reply.
GET_INFO	A5 50	Replies with 20 bytes: model, firmware, hardware, serial number.
GET_HEALTH	A5 52	Replies with 3 bytes: status (0 Good, 1 Warning, 2 Error) and an error code.
SCAN	A5 20	Replies with a short header, then sends measurements forever.

Table 4: The commands. They are defined in `lidar_parse.h`.

Every reply starts with a 7-byte **header** (the official name is *descriptor*). It begins with A5 5A, followed by 5 bytes that tell the length and type of the data. After the **SCAN** command, the header says “length 5, type 0x81”. Then an endless stream of 5-byte pieces follows. We call each piece a **node**. One node is one laser shot.

3.4 What is inside one node

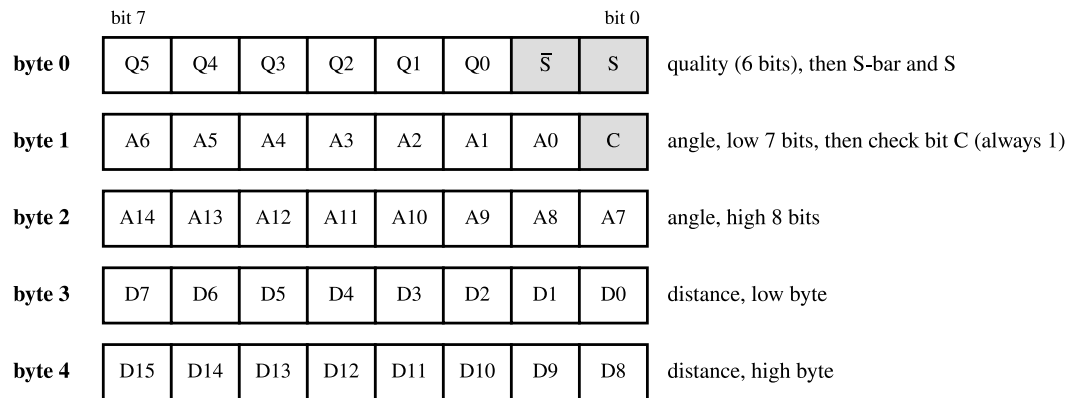


Figure 6: The 5 bytes of one measurement node. The grey bits help us find the start of each node.

- **S** is 1 on the first point of a new turn. **S̄** (S-bar) is always the opposite of S. This gives us a free self-check.
- **C** is always 1. It is another self-check bit.
- **Angle** uses 15 bits. The unit is 1/64 of a degree (we call the value `angle_q6`).
- **Distance** uses 16 bits. The unit is 1/4 of a millimetre (`dist_q2`). A value of zero means “no measurement” (nothing in range, or a bad reading).
- **Quality** is a confidence number from 0 to 63.

3.4.1 A worked example

Imagine the sensor reports: quality 40, angle 90.5°, distance 500 mm, and this is not the start of a turn. We calculate:

- $\text{angle} \times 64 = 5792 = 0x16A0$. $\text{Distance} \times 4 = 2000 = 0x07D0$.
- Byte 0: $(40 \ll 2) \mid (1 \ll 1) \mid 0 = 0xA2$ ($S̄ = 1$, $S = 0$).
- Byte 1: $((5792 \& 0x7F) \ll 1) \mid 1 = 0x41$.
- Byte 2: $5792 \gg 7 = 0x2D$.
- Bytes 3 and 4: $0xD0$ and $0x07$.

On the wire this is `A2 41 2D D0 07`. To decode, you do the same steps backwards. We checked this example with a small script (`check.py`), so you can use it as a test for your own decoder.

TRY IT YOURSELF

Decode `A1 01 00 40 1F` by hand. Is it the start of a turn? What are the angle and the distance? (Answer in Appendix B.)

3.5 One full turn

The sensor never tells us “this turn is finished”. We find it ourselves. When the next node has $S = 1$, the previous node was the last of its turn. So one turn is *all nodes between two $S = 1$ nodes*.

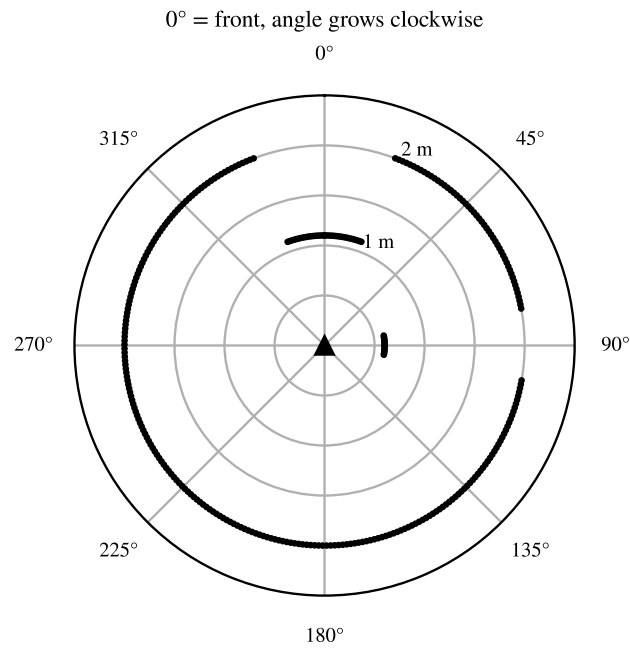


Figure 7: What one turn looks like after we sort the points by angle: a wall at 2 m, a close object at 90°, and something in front at 1.1 m.

CHECK YOURSELF (answers in Appendix B)

Q3.1 Why does the protocol repeat information (S and $\bar{S}$)?

Q3.2 A node says `dist_q2 = 3200`. How many millimetres is that?

Q3.3 How do you know a turn has finished?

4 The Hardware (the FPGA Side)

4.1 Wiring and pins

Lidar	Zybo pin	Port name in the design	FPGA package pin
TX	Pmod JE1	lidar_uart_rxd	V12
RX	Pmod JE2	lidar_uart_txd	W16
MOTOCTL	Pmod JE3	motocctl	J15
GND	Pmod JE5	ground	
VS5.0, VMOTO	external 5 V		

Table 5: The wiring. Look at the crossing: the lidar’s TX goes to the FPGA’s RX.

The port names in the middle must match the names in the constraints file `lidar_zybo.xdc`. This file tells Vivado which physical pin each port uses:

```
set_property -dict { PACKAGE_PIN V12 IOSTANDARD LVCMOS33 } [get_ports lidar_uart_rxd]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports lidar_uart_txd]
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports motocctl]
set_property PULLUP true [get_ports lidar_uart_rxd]
```

Two details are worth understanding:

- **LVCMOS33** tells Vivado that the pin works at 3.3 V. This must match the voltage the Pmod really gets, and the sensor’s signal level.
- **PULLUP true** on the receive pin keeps the line high when nothing is connected. A pin that “floats” picks up noise, and the UART would “receive” random bytes.

THINK OF IT LIKE THIS

The constraints file is the only place where your design touches the real board. A wrong pin number compiles without any error and then fails silently. Always check pins against the board’s official files, as we did with the Digilent board file.

4.2 The block design

The script `hw/build_hw.tcl` builds everything with one command. The block design contains:

1. **Zynq PS7**, with the Zybo preset (memory, clocks, pins), `FCLK_CLK0 = 100 MHz`, and one AXI master port (`M_AXI_GP0`).
2. **AXI UART Lite**: 115200 baud, 8 data bits, no parity. Its pins become the external port `lidar_uart`.
3. **AXI GPIO** with two channels. Channel 1 is an 8-bit output that goes to the PWM block. Channel 2 is a 4-bit output that goes to the LEDs.
4. **motor_pwm**: our own Verilog block, added to the design as a module.
5. An **AXI interconnect** and a **reset block**. Vivado adds these automatically when we connect the helpers.

The script also fixes the addresses (UART Lite at `0x42C00000`, GPIO at `0x41200000`). The C code can then rely on them.

THINK OF IT LIKE THIS

We built the block design with a script instead of clicking in the GUI. This makes it repeatable. A friend with the same Vivado version gets exactly the same hardware by running one command.

4.3 The PWM block, line by line

This is the only hardware code we wrote ourselves. It is short enough to read in full:

```
module motor_pwm #(
    parameter integer CLK_HZ = 100_000_000,
    parameter integer PWM_HZ = 24_000
)()
    input wire      clk,
    input wire      rst_n,
    input wire [7:0] duty,
    output reg      pwm
);
    localparam integer PERIOD = CLK_HZ / PWM_HZ;    // 4166 clocks

    reg [15:0] cnt;
    wire [31:0] thresh = (duty * PERIOD) >> 8;

    always @(posedge clk) begin
        if (!rst_n) begin
            cnt <= 16'd0;
            pwm <= 1'b0;
        end else begin
            cnt <= (cnt == PERIOD - 1) ? 16'd0 : cnt + 16'd1;
            pwm <= (duty == 8'hFF) ? 1'b1 : (cnt < thresh);
        end
    end
endmodule
```

What each part does:

- **PERIOD** is how many clock ticks fit in one PWM cycle. The clock is 100 MHz (10 ns per tick), and PWM runs at 24 kHz. So $100\text{ MHz} / 24\text{ kHz} = 4166$ ticks.
- **cnt** counts from 0 to 4165, then starts again. It is the “counter running around a loop”.
- **thresh** = $\text{duty} \times \text{PERIOD} / 256$. Dividing by 256 is the same as shifting right by 8 bits, which costs no logic at all. This is why the duty is an 8-bit number from 0 to 255.
- The output is high while $\text{cnt} < \text{thresh}$. Duty 0 gives always low. Duty 128 gives 50 %. Duty 120 gives 46.9 % (see figure 5).
- The output is a **register** (**output reg**). It changes only on a clock edge, so it has no glitches.
- **duty == 8'hFF** is a special case. The biggest value the formula gives is $255 \times 4166 / 256 = 4149$. That is smaller than 4166. Without the special case, the output would drop low for a short time (0.4 %) in every period, and would never be fully on.

4.4 Proving it in simulation

Before we used the real board, we tested the block in a simulation. The test file `tb_motor_pwm.v` measures the high time over one full period for several duty values. It compares each result with $\text{duty} \times \text{PERIOD} / 256$. All 7 values pass. You run it with `sim_motor_pwm.bat` in xsim.

THINK OF IT LIKE THIS

If the PWM were wrong on the real board, you would see a motor at a strange speed and have no idea why. A 5-second simulation answers the question before you program the board. Testing a small block alone is cheap. Finding the same bug inside the full system is expensive.

4.5 Timing and the build result

After the build, the worst **setup slack** was **+2.394 ns** at 100 MHz. Slack is the spare time that the slowest signal path has before the next clock tick. A positive number means every signal arrives in time. A negative number means the design is too slow for that clock. A small design like ours passes easily. The number becomes important when your design grows.

CHECK YOURSELF (answers in Appendix B)

- Q4.1 Which line in the constraints file keeps the receive pin high when nothing is connected?
 - Q4.2 How many clock ticks is `motor_pwm` high at duty 200?
 - Q4.3 Why is `duty == 8'hFF` handled separately?
 - Q4.4 Which two lidar wires connect to the FPGA as signals? Which one is only a ground reference?
-

5 The Software, Part 1: Talking to Registers

5.1 How `main.c` is organised

The program has five layers, from the bottom to the top:

1. **Time:** `now_ticks()` and `ticks_to_ms()`, built on the Zynq global timer.
2. **Console:** a printing system that never waits (chapter 6).
3. **Hardware access:** `motor_set()`, `leds_set()`, `lidar_getc()`, `lidar_cmd()`.
4. **Starting the lidar:** info, health, start the scan (chapter 7).
5. **Main loop:** give bytes to the parser, update the LEDs, print, read keys, check the watchdog.

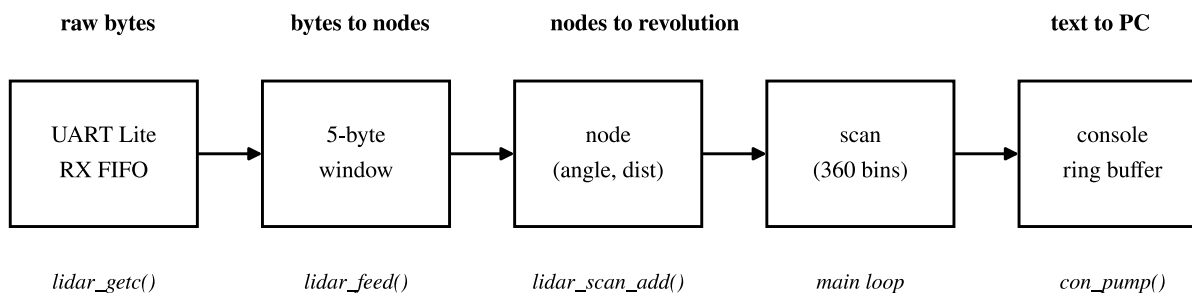


Figure 8: How data moves through the main loop. Every stage handles what is available and returns at once.

5.2 Reading the serial port without waiting

```
static int lidar_getc(void) {
    u32 st = XUartLite_GetStatusReg(LIDAR_UART_BASE);
    if (st & XUL_SR_OVERRUN_ERROR) {
        lidar_overruns++;
    }
    if (!(st & XUL_SR_RX_FIFO_VALID_DATA)) {
        return -1;
    }
    int b = (int)(XUartLite_ReadReg(LIDAR_UART_BASE, XUL_RX_FIFO_OFFSET) & 0xFF);
    ...
    return b;
}
```

First, it reads the status register. If the “overrun” bit is set, the buffer was full at some moment and a byte was lost. The code counts this (reading the status clears the flag). If no byte is waiting, the function returns `-1` immediately. It *never* waits here. Otherwise, it reads one byte.

This method is called **polling**. The main loop keeps asking: “Is there a byte yet?” It is simple, and you have no interrupts to debug. The price is that the loop must come back often enough. That is exactly the topic of chapter 6.

THINK OF IT LIKE THIS

Polling is like checking your mailbox. It works if you check more often than letters arrive, and if the box is big enough to hold the letters that come while you are away. The UART Lite’s box holds only 16 letters.

5.3 Address fallbacks

The Xilinx tools use different names for the address macros in different Vitis versions:

```
#if defined(XPAR_AXI_UARTLITE_0_BASEADDR)
#define LIDAR_UART_BASE XPAR_AXI_UARTLITE_0_BASEADDR
#elif defined(XPAR_XUARTLITE_0_BASEADDR)
...
#else
#define LIDAR_UART_BASE 0x42C00000
#endif
```

The last line is a safety net. If no macro is found, the code uses the address that `build_hw.tcl` assigned. This way, the same code builds in the old Vitis flow and in the new System Device Tree (SDT) flow. The SDT flow changed many names starting from version 2023.2.

5.4 The timer that did not start

This is the best story in the project. It shows what “bare-metal” really means.

The first time we ran the program on the board, the console printed `Motor on, waiting 2 s`. Then nothing happened. Forever. The motor was spinning and the board was alive, but the program was stuck.

All our waiting loops used `XTime_GetTime()`, which reads the Zynq global timer. We found that in the SDT software package, this timer only *starts* inside the first call to `sleep()` or `usleep()`. When you start the board over JTAG (no boot loader), nobody has called these yet. So the timer stays at zero. A loop that waits “until 2000 ms have passed” never ends.

The fix is one line at the top of `main()`:

```
usleep(1000); // starts the global timer as a side effect
```

THINK OF IT LIKE THIS

On bare-metal, nothing runs unless you start it. If your program “hangs”, first suspect a clock or timer that never started, before you suspect your own logic. Toggle an LED or print a message before and after the suspicious loop. See which one never appears.

CHECK YOURSELF (answers in Appendix B)

Q5.1 What does `lidar_getc()` return when no byte is waiting? Why does it not wait?

Q5.2 What does reading the UART Lite status register do to the overrun bit?

Q5.3 Why did the first run freeze at “Motor on, waiting 2 s”?

6 The Software, Part 2: Not Losing Bytes

6.1 The problem

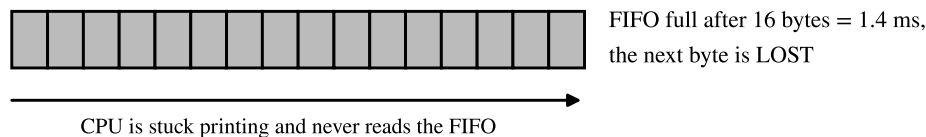
We measured about 1980 nodes per second, 5 bytes each. That is roughly 9900 bytes per second, or one byte every 100 μ s. (The wire can carry at most 11 520.) The UART Lite has a receive buffer (a **FIFO**) of only **16 bytes**. At 115200 baud, 16 bytes arrive in:

$$16 \text{ bytes} \times 10 \text{ bits} \div 115200 \text{ baud} = \mathbf{1.39 \text{ ms}}$$

If the program does not read the buffer within 1.4 ms after it gets full, the next byte is lost. Then the parser sees a damaged node.

Now think about `xil_printf(...)`, the normal print function. It sends one character at a time and waits until the console port can take the next one. The console also runs at 115200 baud. Printing a line of 90 characters blocks the program for about 8 ms. During those 8 ms, nobody reads the lidar buffer. More than 60 bytes arrive in that time, and only 16 fit. So every single print would destroy a large part of a turn.

Blocking print (`xil_printf`)



Non-blocking ring buffer (this project)

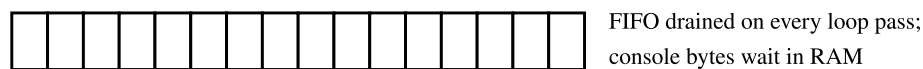


Figure 9: Top: a normal print starves the buffer. Bottom: with a ring buffer, “printing” only puts text in a queue, and the lidar buffer is emptied on every pass.

6.2 The solution: a ring buffer between the program and the console

Instead of printing directly, `con_puts()` copies the characters into a 4 KB array in RAM and returns at once. A second function, `con_pump()`, moves as many bytes as the console port accepts *right now*, and also returns at once. The main loop calls `con_pump()` once per pass, between lidar reads.

```
#define CON_RING_SIZE 4096
static char con_ring[CON_RING_SIZE];
static u32 con_head, con_tail; // head = write, tail = read

static void con_putc(char c) {
    if (con_free() == 0) { con_dropped++; return; }
    con_ring[con_head] = c;
    con_head = (con_head + 1) & (CON_RING_SIZE - 1);
}

static void con_pump(void) {
    while (con_tail != con_head && !XUartPs_IsTransmitFull(CONSOLE_BASE)) {
        XUartPs_WriteReg(CONSOLE_BASE, XUARTPS_FIFO_OFFSET, (u8)con_ring[con_tail]);
    }
}
```

```

        con_tail = (con_tail + 1) & (CON_RING_SIZE - 1);
    }
}

```

Three small ideas make it work:

- **Head and tail.** The writer moves `head` forward. The reader moves `tail` forward. They chase each other around the array like runners on a circular track. The queue is empty when `head == tail`.
- **The size is a power of two.** So going back to the start is just `& (SIZE - 1)`, which is one quick operation instead of a division.
- **Never wait.** If the ring is full, the new byte is thrown away and counted (`con_dropped`). Losing a console character is harmless. Losing a lidar byte is not. So the console is the one that pays.

6.3 The console is also a limit

The short summary line (about 90 characters) is easy. But the optional “frame” line for the PC plot contains 360 distances in hex. It has 1461 characters per turn. At 7 turns per second:

$1461 \times 7.1 \approx 10\,400$ bytes/s, and the console can carry 11 520 bytes/s.

That is already 90 % of what the console can carry. So `print_frame()` first checks `con_free() < 1500`. If there is no room for a whole frame, it skips the frame and counts `frames_skipped`. A skipped frame only means that the plot misses one update. A half-sent frame would spoil the next line.

THINK OF IT LIKE THIS

When two things work at different speeds, put a buffer between them. Also decide in advance what happens when the buffer is full. In this project the rule is: the sensor is more important than the screen, so the screen loses data first.

6.4 Proof, not hope

The code counts lost lidar bytes (`ovr`, from the UART Lite overrun flag) and prints the count on every line. In the console photo in chapter 1, it stays at 0 across hundreds of turns. A claim such as “it does not lose data” is only worth something if the program can show it.

TRY IT YOURSELF

In `main.c`, replace the ring buffer print of the summary line with a normal `xil_printf`. Watch `ovr` go up. Then put it back. (Do this on a copy. Keep the working version safe.)

CHECK YOURSELF (answers in Appendix B)

Q6.1 How long do 16 bytes need to arrive at 115200 baud?

Q6.2 Why is the ring size a power of two?

Q6.3 When all buffers are full, what is dropped first: console text or lidar bytes? Why?

Q6.4 Why does `print_frame()` skip a whole frame instead of sending half of it?

7 The Software, Part 3: Starting the Lidar and the Watchdog

7.1 The start-up steps

Starting the sensor is like a short conversation, and each step can fail. The function `lidar_start()` runs the steps in a loop. It retries forever until they all work:

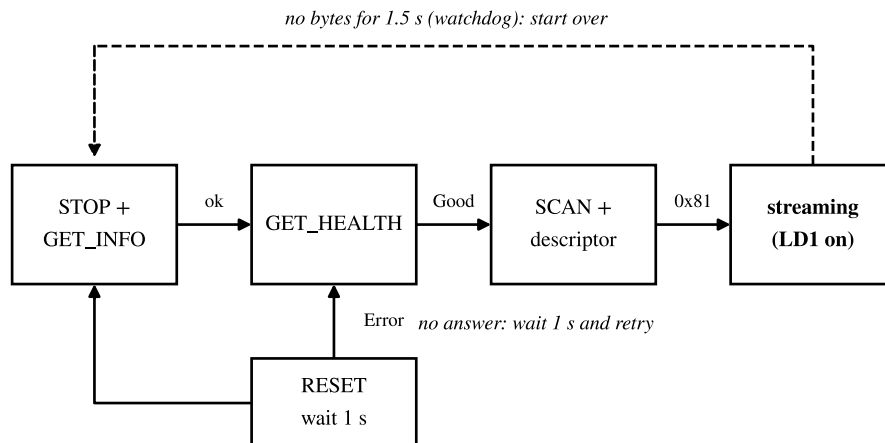


Figure 10: The start-up steps. Solid arrows: the normal path. Dashed arrows: recovery.

1. **STOP and GET_INFO.** Stop any scan that is running, clear the buffer, then ask for the model and serial number. If this works, the link works in *both* directions.
2. **GET_HEALTH.** If it says Good, continue. If it says Error, send RESET, wait one second, and start again.
3. **SCAN.** Wait for the header and check that it says length 5, type `0x81`. Light LD1 and start receiving data.

If the sensor does not answer, the program prints a message that tells you what to check:

```
No answer from lidar (attempt 1). Check: lidar TX->JE1, RX->JE2, GND->JE5, 5 V on VS5.0
bytes received from lidar: 0 (nothing arrives on JE1)
```

It also prints the first raw bytes it received. This helps you tell “nothing arrives” (check wiring and power) from “garbage arrives” (check the baud rate, two drivers on one wire, or a missing ground).

7.2 Time limits with one shared deadline

Every wait in the code uses a **deadline**: a fixed time computed once, then checked again and again.

```
static int lidar_getc_before(u64 deadline) {
    for (;;) {
        int b = lidar_getc();
        if (b >= 0) return b;
        con_pump();
        if (now_ticks() >= deadline) return -1;
    }
}
```

There is a clever point here (it is also written in the source comment). Imagine a noisy line that keeps sending garbage bytes. If we used a separate timeout for each byte, every new byte would restart the timer, and we could wait forever. With one shared deadline for the whole search, the total waiting time is always limited, whatever arrives.

Notice that `con_pump()` also runs inside the wait. Even while we wait for the lidar, the console keeps draining.

7.3 The watchdog

Suppose the data stops while the motor should be running (a cable fell out, or the sensor reset). The program must not just sit there:

```
if (motor_duty && ticks_to_ms(now_ticks() - last_node) > NO_DATA_TIMEOUT_MS) {
    con_puts("No scan data for 1.5 s, restarting\r\n");
    lidar_start();
    ...
}
```

The variable `last_node` is updated on every good node. If it is older than 1.5 s, the program runs the start-up steps again. The part `motor_duty &&` switches the watchdog off when you stop the motor on purpose with the `m` key.

THINK OF IT LIKE THIS
A watchdog is like a dog that barks if nobody feeds it. Every good node “feeds” it. If no food comes for 1.5 s, it barks, and the program restarts the sensor.

7.4 Console keys

Key	What it does
+ / -	Motor duty up or down by 10
m	Motor off, or on again at the earlier duty
f	Turn frame output (for the PC plot) on or off
r	Restart the scan (runs the start-up steps again)
h	Show help

Table 6: Keys you can type in PuTTY while the program runs. They are a fast way to explore the motor speed.

CHECK YOURSELF (answers in Appendix B)

- Q7.1 What two things does a successful `GET_INFO` already prove about the wiring?
- Q7.2 Why is one shared deadline safer than a timeout for each byte?
- Q7.3 Why is the watchdog switched off when the duty is 0?

8 The Parser: Making Sense of the Byte Stream

8.1 The problem

The sensor sends a stream of bytes. There are no markers between the nodes, only the structure inside each 5-byte node. Suppose we start listening in the middle of a node (after a restart, or after a lost byte). Then our alignment is wrong, and every “node” we decode is nonsense. The parser must do three things: notice the problem, throw away bytes until the alignment is right again, and never report a turn it is not sure about.

The parser is plain C with no Xilinx files. So the same file also compiles on a PC and runs inside a test program (sw/host_test).

8.2 Step 1: the sliding window

The function `lidar_feed()` keeps the last 5 bytes in a “window”. It checks them against the rules of the protocol:

```
uint8_t s      = p->win[0] & 1;
uint8_t s_inv  = (p->win[0] >> 1) & 1;
uint8_t c      = p->win[1] & 1;
uint16_t ang   = ((uint16_t)p->win[2] << 7) | (p->win[1] >> 1);

if (s == s_inv || c != 1 || ang >= 360 * 64) {
    // shift the window by one byte and try again
}
```

A window looks like a real node only if $S \neq \bar{S}$, $C = 1$, and the angle is below 360° . If the check fails, we slide the window by one byte: the oldest byte is thrown away and counted in `resync_drops`. If the alignment is right, the next 5 bytes will pass the check again, and again.

8.3 Step 2: one good window is not enough

How likely is it that 5 *random* bytes look like a node? We measured it with a script:

Check	Random bytes that pass
$S \neq \bar{S}$ and $C = 1$ (the two protocol bits)	1 in 4.0
... and the angle is below 360°	1 in 5.7
... and 3 windows in a row pass	1 in about 180

Table 7: How often random data fools the checks (200 000 random 5-byte windows, `check.py`).

A 1-in-4 chance is far too weak. Random noise would fool us all the time. So the parser asks for `LIDAR_LOCK_NODES = 3` good nodes in a row before it trusts the alignment. Windows that pass but have not reached 3 in a row are counted, but not used.

THINK OF IT LIKE THIS

Think of recognising a song. One note fits many songs. Three notes in the right order fit very few. The parser wants three notes before it believes it found the tune.

TRY IT YOURSELF

Run `python check.py` and see which numbers you get. Our first notes said the 2-bit rule passes random bytes “about 1 time in 6”. The script showed 1 in 4 for the two bits alone, and 1 in 5.7 after adding the angle check, so we corrected the README and `lidar_parse.h`. Finding a small mistake in your own notes is a normal part of engineering.

8.4 Step 3: building one turn

After a node is accepted, `lidar_scan_add()` puts it into the turn that is in progress:

- If the node has `S = 1`, the previous turn is finished. We report it *only if* we saw a start before (the first turn is always partial, because we joined in the middle) and it has at least `LIDAR_MIN_REV_PTS = 100` points. A real turn has 200 to 1000 points.
- Nodes with `dist_q2 == 0` count as points, but not as valid measurements.
- The distance in mm is `dist_q2 >> 2`. The angle is rounded to a whole degree with `(angle_q6 + 32) >> 6` (add half a degree, then divide by 64). That number from 0 to 359 is the **bin**, a slot for that degree.
- Each bin keeps the *nearest* distance seen in this turn. The closest one wins, because for safety the nearest obstacle matters most.
- On the way, we also track the nearest point overall (`nearest_mm`, `nearest_deg`) and the nearest point within $\pm 30^\circ$ of straight ahead (`front_mm`).

The main loop turns `front_mm` into LEDs. LD2 turns on if something is closer than 1 m in front. LD3 turns on if it is closer than 30 cm. LD0 toggles every turn (a heartbeat), and LD1 stays on while scanning.

8.5 Tested on the PC first

The test `sw/host_test/test_lidar_parse.c` gives the parser a made-up stream: some garbage first, half a turn, three full turns with different point counts, one byte deliberately dropped in the middle, and finally 100 kB of pure noise. The parser passes 11 of 11 checks. This includes “the noise produces zero turns” and “the dropped byte is recovered”. The board runs the very same `lidar_parse.c`. So a bug found on the PC is a bug fixed on the board, and you can use a normal debugger instead of JTAG.

THINK OF IT LIKE THIS

Put the logic that is easiest to get wrong into a file that does not depend on hardware. Then test it on a PC. This is the cheapest debugging there is.

CHECK YOURSELF (answers in Appendix B)

- Q8.1 A byte is lost in the middle of the stream. About how many bytes does the parser throw away before it is aligned again?
- Q8.2 Why do we keep the nearest distance per degree and not the average?
- Q8.3 Why is the first turn after start-up never reported?
-

9 Test Strategy: Finding Which Half Is Broken

9.1 The test ladder

When you connect two new things for the first time and nothing works, you cannot tell which one is wrong. So we proved each half separately, and only then joined them:

Step	What we did	What it proved
1	Simulated <code>motor_pwm</code> in <code>xsim</code>	The PWM math is right
2	Ran the parser on a fake stream on the PC	The parser logic is right
3	Plugged the lidar into the laptop with its own USB adapter	The sensor works and we understand its protocol
4	Wired Pmod JE2 to JE1 on the Zybo (loopback)	The FPGA's UART sends and receives (we saw <code>A5 50</code> come back)
5	Fake lidar on the laptop, through the Zybo, to the plot	The whole Zybo software works, even with injected bad bytes
6	Real lidar through the laptop, through the Zybo	The parser handles real sensor data
7	Real lidar wired straight to the Zybo	The complete system

Table 8: Seven steps. Each one changes exactly one thing from the step before.

Steps 5 and 6 use the “bridge” mode. With `FAKE_SRC = 1`, the Zybo takes its scan bytes from the console port instead of the lidar port, parses them, and sends the frames back for plotting.

THINK OF IT LIKE THIS

Each step removes one unknown. When step 7 first failed, steps 1 to 6 had already shown that the FPGA, the parser and the sensor were fine on their own. So the problem had to be in the wiring between them.

9.2 The PC tools

File (<code>pc/</code>)	What it is for
<code>rplidar_test.py</code>	Stage 1: talk to the real lidar on a COM port, print info, health and a scan (or use <code>--fake</code>)
<code>lidar_view.py</code>	Live round plot. <code>--lidar</code> (sensor on the PC), <code>--zybo</code> (frames from the Zybo), <code>--bridge</code> (the console bridge), <code>--fake</code>
<code>fake_lidar.py</code>	A software lidar that plays back a simulated room (used in step 5)
<code>rplidar_sniff.py</code> , <code>selftest.py</code>	Raw byte dump for wiring problems; offline checks of the PC code

Table 9: The Python helpers. Install them with `pip install -r pc/requirements.txt`.

CHECK YOURSELF (answers in Appendix B)

Q9.1 Why test the loopback (JE2 to JE1) before connecting the real lidar? What stays the same in the fake-source and real-lidar tests?

10 Build and Run It Yourself

10.1 What you need

- A Digilent **Zybo Z7-10** with its USB cable (power, programming and console in one). Put jumper JP5 on JTAG.
- An **RPLIDAR A1** with its cable and a **separate 5 V USB power source** for it.
- Jumper wires, a breadboard, **Vivado and Vitis 2025.1** with the Digilent board files, and Python 3 (`pip install -r pc/requirements.txt`).

10.2 Step by step

0. Get the code and read the status table in the README.

```
git clone https://github.com/Rennaafi/rplidar-zybo
```

1. Wire it using the table in chapter 4. Check the crossing (lidar TX to JE1, lidar RX to JE2). Connect the grounds. Power the lidar from the separate 5 V source.

2. Test the sensor alone (optional but recommended). Connect the lidar to the laptop with its own USB adapter:

```
python pc\rplidar_test.py COM5
```

3. Build the hardware. Open the “Vivado 2025.1 command prompt” (or run `settings64.bat` first):

```
cd hw  
vivado -mode batch -source build_hw.tcl
```

This creates the project, builds the block design, runs synthesis and implementation, and makes `lidar_zybo.xsa` and `lidar_zybo.bit`. You can also run `sim_motor_pwm.bat` for the PWM test.

4. Build the software. Use the Vitis 2025.1 shell, and use `cmd`, not PowerShell:

```
cd ..\sw  
vitis -s build_sw.py
```

5. Program the board and run. First open a serial terminal on the Zybo’s COM port at 115200. Then:

```
xsdb run_on_board.tcl
```

You should see the banner, **Motor on**, the Model and Health lines, the scan header, and then one `rev ...` line for every turn. Now press `+` and `-` in PuTTY and watch the **Hz** and **pts** columns change.

10.3 Problems we had (so you can avoid them)

- **Do not build in a folder with a space in the path.** Vitis could not create the platform in such a path. We built in a copy under `D:\lidar_build`.
- **In Windows `cmd`, `cd D:\...` from `C:` does not change the drive.** Use `cd /d D:\...` (And use `cmd`, not PowerShell, for Vitis.)
- **Open the serial terminal before you run** (or you miss the banner). Pick the right COM port: the Zybo console and the lidar’s USB adapter use different ports.

11 Results and What They Teach

11.1 The motor speed test

We changed the duty from 40 to 255 with the + and - keys. We wrote down the settled values. `ovr` stayed at 0 the whole time.

Duty	Turns per second (Hz)	Points per turn	Angle between points
40	4.75	417	0.86°
60	5.18	382	0.94°
80	5.61	352	1.02°
120	6.35	312	1.15°
160	6.92	286	1.26°
200	7.30	271	1.33°
240	7.53	263	1.37°
255	7.53	263	1.37°

Table 10: Measured on the real hardware. The default in the code is 120.

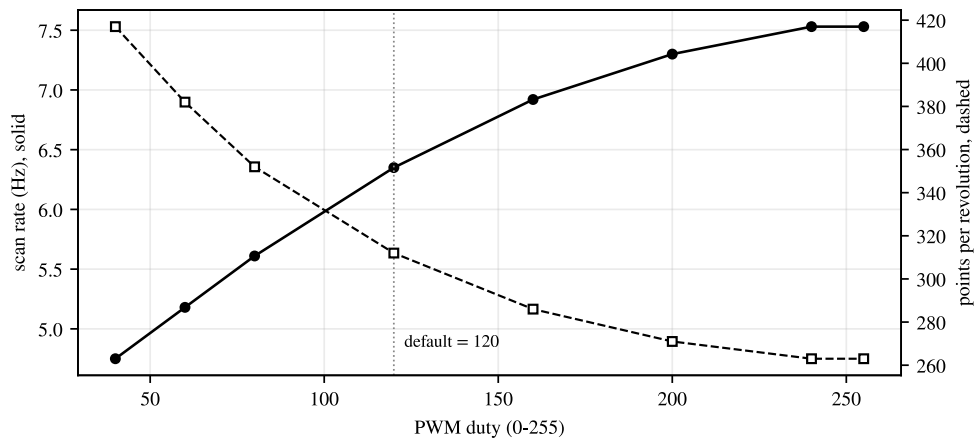


Figure 11: A faster motor gives more turns per second but fewer points in each turn. Above about duty 240, the speed does not go up any more.

11.2 What the numbers tell us

- **Turns per second $\times$ points per turn is always the same.** $6.35 \times 312 \approx 1981$. $7.30 \times 271 \approx 1978$. $4.75 \times 417 \approx 1981$. The sensor takes about **1980 samples per second** whatever the motor speed. A faster motor just spreads the same samples over more turns. So you choose: more updates per second, or more detail in each picture.
- **Above 240 the speed stops growing.** Duty 240 and 255 give the same speed. The motor is already at its limit, so more duty does nothing.
- **We chose 120** as a middle value: 6.35 Hz and 1.15° between points. It leaves room to go faster or slower.
- **Motor off works.** At duty 0 the rate fell from 7.44 Hz to 3.68 Hz within about a second, while the motor slowed down.

A note of caution: we measured that the sample rate is fixed and does not depend on the duty. We did not prove *why* it is 1980. It could be the sensor's own sampling rate. It could also be the limit of the 115200 baud link (at most 2304 nodes per second), which is close. Telling the two apart would be a good experiment.

11.3 Angle between points versus one-degree bins

At duty 120, the laser fires every 1.15° , but the parser keeps one value per whole degree. So “points per turn” (312) is not the same as “valid bins”. Most bins get one point, some get two (we keep the nearer one), and some get none, especially where the sensor found no object. In the console photo you can see `valid` numbers around 160 to 210 from about 280 points. Many laser shots hit nothing within range.

THINK OF IT LIKE THIS

You choose the detail with the motor speed. Slow gives a finer picture of a room that does not change. Fast gives a fresher picture of a moving robot. A mapping robot would probably choose the middle.

CHECK YOURSELF (answers in Appendix B)

Q11.1 Using the “same product” rule, predict the points per turn at 7.0 Hz.

Q11.2 At duty 255, what happens if you press + again?

Q11.3 Why is `valid` lower than `pts`?

12 Bug Diary: What Went Wrong and How We Found It

Real projects are mostly debugging. Here are the real problems, in the order we met them. Each one teaches something.

What we saw	The cause	What it teaches
Program prints “Motor on, waiting 2 s” and stops	The Zynq global timer only starts at the first <code>usleep()</code> . A JTAG start leaves it stopped, so every waiting loop never ends	On bare-metal, check that clocks and timers are started. A one-line fix once you find it.
The Zybo received nothing from the lidar	A USB-TTL adapter’s TX wire was in the same breadboard row as the lidar’s TX. Two outputs fought on one line	Never connect two outputs together. Look at the real wiring, not only the design.
Lidar silent, but wiring matched our guide	Our colour table of the lidar cable had TX, RX and MOTOCTL on the wrong pins. The power wires were right	Check against a photo of the real connector and its printed labels, not only a table.
The lidar misbehaved through the RoboPeak USB adapter	The adapter needs its DTR line released (the PC scripts now do this)	Know what every control line of a helper board does.
Fake “turns” in the PC test	The 2-bit protocol check matches random data far too often	Add range checks and a lock count. Measure the false-alarm rate, do not guess it.
Vitis platform build failed	A space in the project path	Tools have hidden rules. Try the same project in a simple path first.
A driver was missing after the first build	A stale CMake setup. We fixed it by running <code>cmake .</code> then <code>ninja</code> in the BSP build folder	A broken generated build can often be repaired in place. You do not always need to start over.

Table 11: The bug diary. Each row cost us time and taught us something.

THINK OF IT LIKE THIS

Notice that none of these were mistakes in our clever code. They were wiring, timers, paths and tools. The cure was always the same: make the problem smaller until only one thing can be blamed. The test ladder in chapter 9 trains exactly this skill.

13 Exercises

Do them in order. Answers and hints are in Appendix B.

On paper

1. Decode `A1 01 00 40 1F` by hand (the “Try it yourself” in chapter 3).
2. At full line speed, how many bytes arrive at the UART Lite while the console prints one 90-character line at 115200 baud?
3. At duty 200, for how many clock ticks is `motor_pwm` high in one period? What percentage is that?
4. With 1980 samples per second, how many points per turn do you expect at 5 turns per second?

On the board

1. Change `DUTY_DEFAULT` to 200, rebuild, and check that the rate becomes about 7.3 Hz.
2. Change `FRONT_NEAR_MM` to 2000 and see LD2 turn on when your hand is 1.5 m in front.
3. Press `r` in PuTTY while the scan is running. What lines are printed? Which function does this?
4. Unplug the lidar’s TX wire for 3 seconds, then plug it back. What does the watchdog print?

Design

1. Add a key that prints the whole `bins_mm` array once. Where would you put the code? (Hint: the output must go through the ring buffer.)
2. The parser reports the nearest point in the front $\pm 30^\circ$. Change `lidar_scan_add` to report the nearest point in the *rear* $\pm 30^\circ$ too, and show it in the summary line.
3. Sketch how you would move the parser into the FPGA as a Verilog state machine. It reads the UART Lite and writes bins into a block RAM. Which parts would be hard?

THINK OF IT LIKE THIS

If you can do the “Design” exercises on paper, you understood the project. If you can do them on the board, you can build the next one.

14 Where This Goes Next

This guide covers the direct link from lidar to Zybo. That step is complete and tested on the hardware. Here are natural next steps, from easiest to hardest:

- **Use the data.** Convert each bin to x and y ($x = r \sin \theta$, $y = r \cos \theta$, with θ measured clockwise from the front). Add up the scans while the robot moves. This is the start of mapping, and a natural input for the Indoor Mapping Robot project.
- **Move the parser into the FPGA.** A Verilog state machine reads the UART Lite stream and writes bins into block RAM. This frees the ARM and teaches you real state-machine design.
- **Send the scans over a network.** Replace the USB console with a network link, so another machine can show the picture. The Ethernet work with the ESP is a separate stage and is not covered here.
- **Use interrupts instead of polling,** and add DMA, when the polling loop gets too busy.

THINK OF IT LIKE THIS

Each step reuses something from this project: the parser, the ring-buffer idea, the test ladder. This is the sign of a good small project. It leaves you parts that you can keep.

Appendix A: Glossary

AXI	The “road” the PS uses to talk to parts in the PL. Here the ARM is the boss and the PL blocks are helpers.
Baud	Symbols per second on a serial line. At 115200 baud with 8N1 that is 11 520 bytes per second.
Bare-metal	Software that runs with no operating system.
Bin	One of 360 slots, one per degree, holding the nearest distance.
BSP	Board support package: the low-level drivers generated for your hardware.
Duty cycle	The fraction of a PWM period during which the output is high.
FIFO	A buffer where the first byte in is the first byte out. The UART Lite receive FIFO holds 16 bytes.
JTAG	The connection used for programming and debugging (here over the same USB cable).
Node	One 5-byte lidar measurement: quality, angle, distance, start flag.
Overrun	A byte arrived when the FIFO was already full, so it was lost.
PL / PS	Programmable logic (the FPGA part) / processing system (the ARM part).
Pmod	Digilent’s 12-pin connector for add-on parts. Pins 1-4 are signals, 5 is ground, 6 is 3.3 V.
PWM	Pulse-width modulation: switching an output on and off to set an average level.
Resync	Throwing bytes away until the parser lines up with the node boundaries again.
Ring buffer	A fixed array used as a queue. Its head and tail positions wrap around to the start.
SDT	System Device Tree: the newer Xilinx flow (version 2023.2 and later) for generating the BSP.
Slack	Spare time, in nanoseconds, that a signal path has before the next clock tick. Positive is good.
WNS	Worst negative slack: the smallest slack in the whole design. Ours: +2.394 ns.

Appendix B: Answers

Chapter 1. (1) PS (processing system, the ARM) and PL (programmable logic, the FPGA part). The PS runs `main.c`. (2) Verilog: `motor_pwm.v` and its test. C: the main program and the parser. (The UART Lite and GPIO are ready-made blocks.)

Chapter 2. (1) 8.68 μ s per bit, so 86.8 μ s per byte (10 bits). (2) The 8 outputs of GPIO channel 1 change. They feed `motor_pwm`, whose output is the MOTOCTL pin (JE3). (3) `sleep` blocks the program, so nothing else can read the serial port, and the 16-byte FIFO would overflow.

Chapter 3. The “Try it yourself” decode: `A1 = 1010 0001`. `S = 1` and `S̄ = 0` (valid), so yes, it is the start of a turn. `Quality = 0xA1 >> 2 = 40`. Byte 1 = `0x01` gives `C = 1` and low angle bits 0. Byte 2 = `0x00`, so the angle is 0°. `Distance = 0x1F40 = 8000`, divided by 4 = 2000 mm. (1) So errors can be detected and a wrong alignment can be noticed. (2) `3200 / 4 = 800` mm. (3) When the next node has `S = 1`.

Chapter 4. (1) `set_property PULLUP true [get_ports lidar_uart_rxd]`. (2) `200 × 4166 >> 8 = 3254` ticks, 78.1 %. (3) The biggest threshold from the formula is 4149, less than 4166. Without the special case it would never reach 100 %. (4) TX and RX are signals. GND is the reference.

Chapter 5. (1) −1. If it waited, the loop could not do anything else. (2) Reading it clears the overrun bits, so each event is counted once. (3) The timer had not started. `usleep` starts it.

Chapter 6. (1) 1.39 ms. (2) Wrapping around becomes `& (SIZE - 1)`, which is one quick operation. (3) Console text. Losing text is harmless, but losing lidar bytes damages the data. (4) A half frame would spoil the next line that the plot program reads.

Chapter 7. (1) The lidar’s RX receives our command, and its TX reaches us (both directions work). A common ground is also there. (2) Noise bytes would restart a per-byte timer forever. A shared deadline always ends. (3) With the motor off, no data is expected, so restarting would loop for nothing.

Chapter 8. (1) At most 4 bytes are thrown away before the window is aligned again (the `sync` counter on the console shows them). Then 3 good nodes (15 bytes) are needed before reporting continues. (2) The nearest point is what matters for safety. An average would hide a close obstacle. (3) It is partial. We joined it in the middle.

Chapter 9. (1) The loopback proves the FPGA’s UART works without the lidar. Otherwise you cannot tell if the lidar or the FPGA is wrong. The Zybo parser and the plot stay the same in all tests. Only the byte source changes (fake room, real lidar through the laptop, real lidar direct).

Chapter 11. (1) `1980 / 7.0 ≈ 283`. (2) The duty stops at 255 (the code limits it). (3) Many laser shots return no distance (zero). They count as points but not as valid points.

Chapter 13, on paper. (2) `90 characters × 10 bits = 900 bits ≈ 7.8 ms`. At the full 11 520 bytes/s, about 90 bytes arrive during one print (about 77 at the measured 9900 bytes/s). The FIFO holds only 16. (3) `3254` ticks, 78.1 %. (4) `1980 / 5 = 396` points.

Chapter 13, board and design (hints). Board 1: the speed table says duty 200 gives 7.30 Hz. Board 2: `FRONT_NEAR_MM` is near the top of `main.c`. The LED logic is in the main loop. Board 3: it prints `restarting scan`, then the Model, Health and header lines, all from `lidar_start()`. Board 4: `No scan data for 1.5 s, restarting`, followed by the start-up lines. The watchdog only runs while the duty is above 0. Design 1: add a `case` to `handle_key()` that loops over `bins_mm` and calls `con_puthex`. Do not use `xil_printf` (chapter 6). Design 2: copy the front test in `lidar_scan_add`, use a window around 180°, add a `rear_mm` field to `lidar_scan_t`, and print it in `print_summary`. Extend `host_test` to check it. Design 3: the hard parts are the alignment search (a 5-byte shift register plus a lock counter), the bin memory update (read, compare, write on a block RAM), and the handshake with the ARM when a turn is complete.

Appendix C: Quick Reference

Board	Digilent Zybo Z7-10 (xc7z010clg400-1)
Sensor	RPLIDAR A1 (A1M1-R1), UART 115200 8N1, 3.3 V signals, 5 V power
Pins	JE1 = V12 (lidar TX), JE2 = W16 (lidar RX), JE3 = J15 (MOTOCTL), JE5 = GND
PWM	24 kHz, 8-bit duty, <code>thresh = duty × 4166 >> 8</code>
Addresses	UART Lite <code>0x42C00000</code> , GPIO <code>0x41200000</code> , console UART1 <code>0xE0001000</code>
LEDs	LD0 heartbeat per turn, LD1 scanning, LD2 object closer than 1 m in front, LD3 closer than 30 cm
Default	Duty 120: 6.35 Hz, 312 points, 1.15° between points
Timing	WNS +2.394 ns at 100 MHz
Parser	3 nodes to lock, at least 100 points per turn, 360 one-degree bins
Console	4 KB ring buffer. A frame line (1461 characters) is skipped when fewer than 1500 bytes are free
Tests	<code>check.py</code> (numbers), <code>sw/host_test</code> (parser, 11/11), <code>hw/sim_motor_pwm.bat</code> (PWM, 7/7)